

TruthLens: A Verification-Gated Pipeline for Evidence-Grounded Fact-Checking of Short-Form Political Video

Aditya Rekhe

[Affiliation placeholder – TODO]

Email: adityarekhe1030@gmail.com

Abstract—Short-form video is now a primary vector for political misinformation, yet most automated fact-checking research targets text – tweets, news sentences, or curated claim lists – and most LLM-based fact-checking systems are evaluated on whether they reach a correct final label, not on whether the intermediate evidence chain that produced that label is real. We present TruthLens, a fact-checking pipeline for short-form political video (Instagram Reels, in the deployed case study) that enforces a strict evidence-before-verdict ordering – atomic claim decomposition, multi-query web research, tiered source retrieval, per-source evidence extraction, and only then a claim-level verdict – and, distinctively, subjects every LLM-generated verdict to a deterministic, non-LLM validation pass before it can be published. The validator checks that every cited piece of evidence actually exists, that every cited source was actually fetched, and that every number asserted in the model’s reasoning actually appears in a cited passage; verdicts that fail are automatically downgraded rather than silently trusted. We further introduce an explicit distinction between UNVERIFIED (evidence was genuinely sought and found insufficient) and RESEARCH_FAILED (the retrieval infrastructure itself broke), so that an infrastructure failure can never be published as if it were a considered null result, and a cost-aware model-escalation pattern in which a free, locally-hosted model is used by default and a paid model is invoked only when a deterministic “is this output substantively grounded” check fails. We report preliminary operational telemetry collected during iterative development – including that deterministic validation downgraded 28.6% (6 of 21) of generated verdicts in our test corpus, and that the free local model required escalation on 25–43% of invocations depending on pipeline stage – and are explicit about what these numbers do and do not establish: they come from a small, non-independent, single-domain test corpus accumulated during debugging, not a held-out benchmark. A small first step toward a real held-out check – two reels TruthLens had never seen, each independently fact-checked by a professional Indian fact-checking organization – found its verdict agreeing with the independent ground truth in both cases, but also surfaced a more consequential result: on both, the system’s text-derived claim extraction missed the actual visual misinformation (real footage recaptioned with a false claim about what it shows), and one run exposed a real gap in validation coverage that let a downgraded verdict’s hallucinated reasoning reach a rendered slide, which we traced and fixed the same day. A first, deliberately narrow naive-baseline comparison on the same two claims found the baseline abstaining (correctly recognizing it had no way to know about recent events) where TruthLens’s pipeline reached a substantive, correct-at-the-label answer – a result we are careful to attribute to search access rather than to any demonstrated advantage in TruthLens’s specific architecture, pending

a search-equipped baseline. Extending ingestion to single-image posts (not just video) surfaced a fabricated headline number in the process – “4,000 crore rupees” rendered as an invented, and arithmetically wrong, “\$1 Billion” – caught only after adding a dedicated deterministic check, since the model violated an explicit prompt instruction not to introduce new numbers. We further close a gap our own evaluation flagged as the most consequential open problem for output quality: retrieval was rarely reaching primary government/legal sources despite prompt-level requests for them (8 of 72 sources, ~11%, in our initial corpus); enforcing a domain allow-list at the search-call level itself, the same mechanism already used for fact-checking-organization sources, raised this to 19 of 20 (95%) across live-tested queries, while surfacing an honest residual limitation of its own – unreliable full-page-text retrieval specifically on .gov.in domains, left unresolved. We also extend the validation methodology beyond verdict labels to grounded corrections: when a claim is not simply true, TruthLens now surfaces what the same evidence establishes instead, held to the identical grounding standard, while deliberately not attempting the further step of predicting downstream consequences, which would require exactly the kind of ungrounded generation this paper argues against. A third live test found two further real bugs outside the LLM reasoning stages entirely: claim extraction silently accepting schema-valid but empty-string claims, and a provider wrapper’s error handling written against the wrong exception hierarchy for the API surface actually in use, which let a routine rate-limit condition crash an in-flight request after roughly forty minutes instead of failing fast – notably, a pre-existing unit test for that exact code path had mocked the same wrong exception type the production code mishandled, passing throughout despite providing no real coverage. Both are fixed and reverified live, though this specific post did not yield a finished fact-check, a mixed result we report rather than omit. We describe the larger evaluation this motivates and a taxonomy of concrete failure modes discovered through live testing that we believe generalize to other LLM-pipeline fact-checking systems.

Index Terms—automated fact-checking, large language models, hallucination mitigation, retrieval-augmented generation, misinformation detection, short-form video, claim verification, source credibility

I. Introduction

Political misinformation increasingly spreads through short-form video – Instagram Reels, YouTube Shorts, TikTok – rather than text [1]. This shift matters for automated fact-checking research in two ways that are easy to underestimate. First, the input is multi-modal and

lossy: a claim may be spoken (transcript), displayed as on-screen text (OCR), or only implied by a caption, and any system that treats a reel as a single block of text discards exactly the structure a fact-checker needs to attribute who said what. Second, and more consequentially, the dominant existing evaluation paradigm for automated fact-checking – did the system output the correct veracity label for a claim, measured against a benchmark such as FEVER [2] or LIAR [3] – says nothing about whether the evidence the system cites in support of that label is real. An LLM asked to “fact-check this claim” can produce a fluent, well-cited-looking verdict whose citations point to sources it never retrieved, or whose numbers never appeared in any source it read. This is the specific failure mode this paper is about.

We built and iteratively debugged TruthLens, a pipeline that ingests a political short-form video, decomposes it into atomic factual claims, researches each claim against the open web, and produces a carousel-style, cited, claim-by-claim fact-check – while trying to make that specific failure mode (correct-sounding but ungrounded output) structurally difficult rather than merely discouraged by prompting. The central design principle, stated as a hard constraint throughout development, is evidence before verdict: the system is not permitted to propose a verdict and then search for support; it must search first, and a verdict can only be built from evidence rows that already exist in the database at the time the verdict-proposal model is invoked.

Enforcing this ordering architecturally is necessary but not sufficient, because even a model given only real, already-fetched evidence can still hallucinate – inventing a citation to evidence that was never actually supplied, asserting a number the source never stated, or (a failure mode we discovered live, not anticipated in advance) padding a `reasoning_summary` field entirely with internal citation markup and no actual prose, which is schema-valid (a non-empty string) while being substantively empty. TruthLens’s second contribution is therefore a deterministic, non-LLM validation layer that re-checks every LLM output against ground truth already in the database before it is allowed to reach a reader, downgrading rather than trusting outputs that fail.

This paper makes the following contributions:

- 1) An architecture and reference implementation for evidence-grounded, multi-claim fact-checking of short-form video, including atomic claim decomposition with verbatim-quote grounding, multi-tier source credibility scoring, and deterministic (non-LLM) aggregation of per-claim verdicts into a single auditable reel-level verdict (Section III).
- 2) A deterministic post-hoc validation methodology – citation existence, source-fetch existence, and number-grounding checks – that catches a measurable fraction of ungrounded LLM output without requiring a second, equally fallible LLM call to police

the first (Section IV).

- 3) An explicit `RESEARCH_FAILED` vs. `UNVERIFIED` distinction, and a publication gate that refuses to let infrastructure failure masquerade as a considered null result – an instance of the broader selective-prediction principle that a system should distinguish “I looked and found nothing” from “I could not look” (Section IV).
- 4) A cost-aware escalation pattern – default to a free, local model; escalate to a paid model only when a deterministic substantiveness check fails – evaluated, as preliminary telemetry, on how often escalation actually triggers per pipeline stage (Section V).
- 5) A concrete taxonomy of failure modes discovered through live, iterative testing against a real deployed reel, several of which (a paraphrase misattributed as a direct quote; a dashboard verdict that silently disagreed with the published carousel verdict; a bracket-matching validation-note regex that truncated at the wrong bracket) are not, to our knowledge, previously documented in the automated fact-checking literature at this level of implementation detail (Section VI).
- 6) An extension of the validation methodology from verdict labels to grounded corrections and context – what the evidence establishes instead of a false or unverified claim, held to the same deterministic grounding standard as the verdict itself – together with an explicit, motivated decision not to extend it further into predicting consequences, which would require ungrounded generation of exactly the kind this paper’s architecture exists to prevent (Section IV-E).

We are explicit throughout about the evidentiary status of our claims. Section V reports real, measured telemetry from TruthLens’s own audit log, not simulated or projected numbers – but that telemetry comes from a small corpus (three distinct source videos, 16 distinct claims, 21 verdict-generation events including re-runs across pipeline-fix iterations) accumulated during debugging rather than a frozen, held-out benchmark, and we describe in Section VIII the larger evaluation required before any accuracy claim here should be treated as representative.

II. Related Work

Automated fact-checking and LLM factuality evaluation have both grown into large enough literatures to warrant their own surveys [4], [5]; we focus below on the specific sub-areas TruthLens draws on and departs from, rather than attempting a comprehensive survey ourselves.

A. Claim detection and decomposition

Claim detection for fact-checking spans monolingual, multilingual, and cross-lingual settings [6], all of which are relevant to TruthLens’s mixed English/Hindi/Urdu input. ClaimBuster [7] established check-worthy claim detection

as a distinct pipeline stage, using a TF-IDF/POS/NER-featured SVM to rank sentences from political debate transcripts by how worth fact-checking they are. TruthLens’s claim-extraction stage performs a related but distinct task: rather than ranking existing sentences, an LLM decomposes a transcript, OCR stream, and caption into new atomic claims, splitting compound statements (e.g. “X happened, and because of it Y happened”) into independently checkable parts, following the general claim-decomposition strategy used in systems such as ProgramFC, which decomposes complex claims into sub-questions before per-question retrieval. We extend this with a grounding requirement specific to video: each extracted claim’s `source_quote`, when present, must be a verbatim substring of the actual transcript/OCR text, checked deterministically, not merely asserted by the model.

B. Fact-checking benchmarks

FEVER [2] (Wikipedia-derived claims, Supports/ Refutes/NotEnoughInfo) and LIAR [3] (12.8K PolitiFact statements, six-way truthfulness) are the dominant benchmarks for claim-verification models, and both evaluate a system’s final label against ground truth without examining whether the evidence chain that produced the label is real. TruthLens’s evaluation gap (Section VIII) is precisely that we do not yet have a FEVER/LIAR-scale held-out benchmark for our own domain (Indian political short-form video); closing that gap, likely by curating a small labeled set in the style of these benchmarks rather than reusing them directly (our claims arrive multi-modally and in Hindi/Urdu/English mixed transcripts, which neither benchmark covers), is the single most important item of future work.

C. LLM hallucination and grounding verification

A growing body of work targets citation and grounding hallucination in LLM output specifically. A preregistered, hand-scored evaluation of leading commercial legal AI research tools found each hallucinates on 17–33% of queries [8], and recent systems such as CiteCheck [9] detect hallucinated references by grounding each citation against external sources with a structured LLM verifier. Our validation layer (Section IV) shares the goal but differs in mechanism: rather than using a second LLM call to judge whether a citation is plausible, TruthLens’s checks are deterministic set-membership and substring tests against data the pipeline itself already fetched (does the cited `evidence_id` exist; was the cited source actually retrieved; does the asserted number appear in the cited passage’s stored text). This trades generality (we cannot catch a hallucination that happens to reuse a real, cited evidence ID incorrectly) for auditability and zero marginal LLM cost per check. A recent analysis of citation-grounding oracles in the legal domain is a useful caution for exactly this kind of check: it shows that an apparent hallucination

rate measured by verifying citations against a reference graph is highly sensitive to that graph’s coverage, not just to the model being measured, with the same responses scoring 15–21% “hallucinated” against a sparse snapshot of a citation registry and under 1.1% against a denser one built from the same source [10]. TruthLens’s own number-grounding check has an analogous blind spot: it can only confirm a number against sources the pipeline itself fetched, so it cannot distinguish “this number is fabricated” from “this number is real but supported only by a source we did not retrieve.” We flag this as an open limitation rather than a solved problem.

D. Political fact-checking with LLMs

Most directly relevant to TruthLens’s domain, DeVerna et al. [11] evaluated 15 LLMs against over 6,000 PolitiFact-verified claims and found that reasoning ability provided “minimal benefit” and raw web search only “moderate gains” for political fact-checking accuracy, while providing curated, high-quality context improved macro-F1 by an average of 233% across models. This is strong independent evidence for TruthLens’s core design bet: that pipeline-level evidence curation (multi-query research planning, source tiering, per-source relevance filtering) matters more than which LLM does the final reasoning step. We view TruthLens as, in part, an applied instantiation and extension of that finding to a harder, multi-modal input domain.

E. Short-form video misinformation

TikTec [1] detects misleading COVID-19 short videos on TikTok by jointly modeling visual, audio, and textual signal, using captions to focus attention on the most informative frames. Recent work on checkworthiness detection specifically targets multilingual short-form video [12]. TruthLens differs from this line of work in task framing: rather than a binary/graded misinformation classifier over the video as a whole, TruthLens performs open-domain, claim-level, cited verification against live web evidence, closer in spirit to FEVER-style verification than to TikTec-style classification, but applied to video-derived claims rather than Wikipedia sentences.

F. Political neutrality of automated fact-checkers

LLMs are not neutral instruments; recent evaluations document measurable partisan asymmetries in frontier models under politically charged prompts [13]. Because a visibly biased automated fact-checker is worse than none for a politically sensitive application, every TruthLens system prompt includes an explicit neutrality clause instructing the model to apply identical evidentiary standards regardless of which political actor is named and to be equally willing to conclude a claim is true or false for any side. We have not run a systematic bias audit of TruthLens’s own output (see Section VIII); this is a limitation, not a resolved claim.

G. Cost-aware model cascading

FrugalGPT [14] demonstrated that routing most queries to a cheap model and escalating only hard cases to an expensive one can sharply cut inference cost with little accuracy loss. TruthLens applies the same principle in reverse-priority form suited to a zero-marginal-cost constraint: every stage defaults to a free, locally hosted model (Llama 3.2 via Ollama) rather than a cheap paid one, and escalates to a paid model (Gemini) only when a deterministic quality gate – not a learned cost/accuracy predictor as in FrugalGPT – detects that the local model’s output is schema-valid but substantively empty.

III. System Architecture

TruthLens is a FastAPI backend with a PostgreSQL/pgvector datastore, a Next.js review dashboard, and an object store (S3-compatible) for media and rendered output. The pipeline runs in the following strict order for every reel, and no stage may begin before the stage before it has persisted its output to the database:

- 1) Ingestion. The source video is downloaded (or manually uploaded), audio is extracted and transcribed, on-screen text is sampled and OCR’d at a fixed interval, and a representative thumbnail frame is selected. A single-image (photo) post follows a parallel path with no video stream at all (Section III-C).
- 2) Claim extraction. An LLM decomposes the transcript, OCR text, and caption into atomic claims, each classified as factual, opinion, prediction, satire, or rhetorical, with only factual claims specific enough to research marked verifiable.
- 3) Research planning. For each verifiable claim, an LLM proposes 2–5 targeted search queries, explicitly instructed to include at least one query aimed at primary sources (official/government domains) when plausible.
- 4) Search and fetch. Each query is run against a search provider; result pages are fetched and their full text extracted (not just the search snippet).
- 5) Evidence analysis. For each fetched source, an LLM judges stance (supports/contradicts/provides context/irrelevant) against the claim, grounded in that source’s actual extracted text only.
- 6) Verdict proposal. Given the claim and its full evidence matrix (every evidence row, with source reliability and stance), an LLM proposes a verdict label, confidence, and cited evidence IDs.
- 7) Deterministic validation (Section IV).
- 8) Overall-verdict aggregation. Once every claim covered by a fact-check has a validated verdict, a fixed rule table – not an LLM call – derives one reel-level verdict.
- 9) Content assembly and rendering. A carousel of slides (headline poster, claims/quote, evidence, conclusion) is assembled, drawing almost all content directly from already-validated database rows; only headline

phrasing and a short “why” paragraph pass through one additional narrow, schema-constrained LLM call each, both with their own grounding checks (Section IV).

A. Source tiering

Every retrieved source is scored along an 8-dimension weighted reliability rubric and assigned to one of eight tiers: `primary_government`, `primary_legal`, `primary_data`, `news_wire`, `established_news`, `academic`, `factcheck_org`, or other. Tier membership determines both which sources are eligible to be surfaced as a claim’s “primary source” citation on the rendered carousel and, in aggregate, the reliability-weighted confidence of the evidence matrix handed to the verdict-proposal stage.

B. Multi-claim, reel-level verdicts

An earlier version of TruthLens built one fact-check per claim; the current version builds one fact-check per reel, covering every verifiable claim extracted from it, because a single short-form video routinely makes several claims of differing veracity, and collapsing them into one per-claim verdict discards exactly the “partially true, partially misleading” structure a reader needs. The rule table in Section IV governs how per-claim verdicts combine into the one verdict actually shown on the published carousel.

C. Multi-modal ingestion: video and photo posts

Not every short-form political post is a video. TruthLens originally handled only content with a video stream; a real post (Section V-I) exposed this as a hard failure rather than a degraded case. We added a parallel photo-post ingestion path: when the video downloader reports no video stream at all, the pipeline falls back to the post’s Open Graph metadata (image URL, caption, engagement counts) fetched directly from the page rather than through the video-download tool, which does not expose a photo-only code path. A new `media_type` field (video/photo) drives branching at exactly one point downstream: audio extraction and transcription are skipped for a photo (there is no audio), but OCR and vision-context analysis needed no new code at all, because both already operated on a plain list of image paths rather than assuming a video – a single photo is a one-element case of the same interface a video’s sampled frames already satisfy. This is a small but relevant point for system design in this domain: designing the frame-analysis interface around “a list of images” rather than “frames extracted from a video” made a later, unplanned input modality nearly free to add.

IV. Methodology: Making the Model More Trustworthy, Not Just More Capable

The paper’s title question – how to make the model “really better” for this purpose – is, in our experience building TruthLens, substantially a question about the pipeline around the model rather than the model itself.

This section describes the four mechanisms we found to matter most.

A. Deterministic anti-hallucination validation

After the verdict-proposal LLM call returns a structured VerdictProposal (label, confidence, reasoning, cited evidence IDs), TruthLens runs three checks against data already in the database, none of which invoke an LLM:

- Citation existence: every ID in `cited_evidence_ids` must correspond to an evidence row that was actually passed to the model in its evidence matrix.
- Source-fetch existence: every cited evidence row’s source must have a stored, successfully fetched full-text passage — a citation cannot point at a source the pipeline only planned to fetch.
- Number grounding: every numeral appearing in the model’s `reasoning_summary` is extracted and checked for appearance in at least one cited source’s stored passage text.

A verdict that fails any check is not discarded – it is downgraded: persisted with a `validation_status` other than passed, an appended, clearly delimited [VALIDATION NOTE: ...] explaining what failed, and excluded from citation lists shown to a reader. This preserves the model’s underlying verdict label (still potentially useful to a human reviewer) while refusing to let its specific, unverified justification reach a reader unflagged.

B. The RESEARCH_FAILED / UNVERIFIED distinction

UNVERIFIED is a legitimate, informative output: it means the system genuinely searched and found insufficient evidence to support or refute a claim. Early in development, TruthLens conflated this with a second, very different situation – the search backend itself returning zero usable results because of a missing API key or provider misconfiguration – and both surfaced identically as UNVERIFIED, indistinguishable to a reader from a considered null result. We introduced a distinct RESEARCH_FAILED claim status, raised only when all queries for a claim fail at the infrastructure level (not when queries succeed but return no relevant results), and a publication gate that refuses to build a fact-check whose primary claim is RESEARCH_FAILED. This is a direct, domain-specific instance of the broader selective-prediction principle that a system should distinguish “I don’t know” from “I could not check”.

C. Verification-gated model cascading

Every pipeline stage defaults to a free, locally hosted model. Three stages – claim extraction, content generation (headline/why-paragraph), and verdict proposal – additionally run a deterministic substantiveness check on the local model’s output before accepting it:

- Claim extraction, grounding: is the extraction grounded, i.e. do at least half of the extracted

verifiable claims have a `source_quote` that is an actual substring of the combined transcript/OCR/caption text?

- Claim extraction, substantiveness: does at least one extracted claim have non-empty text? Added after a real failure found live (Section V-J): a locally hosted model returned claims with `text=""` for content that plainly had checkable claims in it – schema-valid, but with nothing for the rest of the pipeline to act on, and invisible to the grounding check above, which only ever evaluates verifiable claims and is a no-op when every returned claim is non-verifiable, exactly what happened here. An extraction with zero claims at all is not treated as a failure by this check – a genuine “found nothing” is a legitimate outcome, not a quality problem to retry.
- Content generation / verdict reasoning: after stripping internal citation markup ([`evidence_id=...`] blocks), does the remaining prose contain at least six real words? This check exists because we discovered, live, that a locally hosted model would sometimes produce a `reasoning_summary` composed entirely of citation markup with no prose at all – schema-valid (a non-empty string), but explaining nothing to a reader.

On failure, the same call is retried once against a stronger paid model (Gemini) with an identical prompt, and the retry (whichever model produced it) is logged. This differs from a classic cascade [14] in that the escalation trigger is a deterministic grounding/substantiveness predicate specific to this domain’s failure modes, not a learned or generic confidence estimate.

D. Deterministic overall-verdict aggregation

Once every claim covered by a fact-check has a verdict, TruthLens derives one reel-level verdict via a fixed rule table over the set of per-claim verdict labels (weighted by whether a claim is “major,” importance ≥ 0.5) – for example, all claims TRUE yields overall TRUE; a mix containing both TRUE/MOSTLY_TRUE and FALSE yields MISLEADING. This is a plain combinatorial function, not a further LLM call, so the single highest-stakes number in the product – the verdict shown on the published carousel’s first slide – remains fully auditable from already-validated per-claim verdicts rather than resting on a second, opaque judgment.

E. Grounded corrections: what the evidence shows instead

A verdict of FALSE or UNVERIFIED alone is a weak deliverable – professional fact-checkers do not stop at “this is false”; they say what is actually true, with a source. We extended the verdict-proposal call (rather than adding a new one) with two optional fields, `corrected_fact` and `context_note`, populated from the same evidence matrix already given to that stage and held to the identical “nothing not already in the evidence” rule as

reasoning_summary. corrected_fact is the specific alternative fact the evidence establishes (a different number, a different date), left null rather than guessed when no such alternative is established; context_note is background the evidence itself states, never the model’s own speculation.

We deliberately did not build the natural next request – speculation about downstream consequences of a claim. Predicting what a false or misleading claim about a real political actor might lead to is forward-looking generation with no evidence to ground it against, which is exactly the failure category Section IV-A’s architecture exists to prevent. Scoping it out is a design decision, not an oversight, and we note it here because the boundary between “grounded context” (safe, and built) and “predicted consequences” (ungrounded, and not built) is precisely the line this paper’s whole argument is drawn along.

Both fields are validated independently of reasoning_summary, against numbers found anywhere in the claim’s full evidence matrix rather than only the sources cited for the main verdict – a correction can legitimately draw on a different source than the one that determined the label. This independent check uses every digit in a number, not just those with two or more digits as Section IV-A’s original number check does: that threshold exists to avoid flagging incidental meta-numbers in longer prose (“we reviewed 3 sources”), but corrected_fact and context_note are short, specific, single-sentence fields where no such incidental number should appear, and a single-digit fabrication (Section VI’s “\$1 Billion”) would otherwise slip through uncaught. Finally, whenever the parent verdict fails validation, neither field is trusted for display even if it individually passed its own check – the same “don’t reuse anything from an already-unreliable call” rule already applied to reasoning_summary (Section VI).

V. Evaluation

A. What this section is, and is not

We report real telemetry recorded in TruthLens’s own audit log during iterative development and live debugging against a small corpus: five ingested reel records over three distinct underlying videos (by content hash), yielding 28 extracted claims (16 marked verifiable and carried through to a verdict at least once), and 21 verdict-generation events in total – more than the number of distinct claims because several claims were re-verdicted after pipeline fixes across iterations. This is operational telemetry from a system under active development, not a held-out benchmark evaluation: the sample is small, the units are not independent (the same claim appears multiple times across pipeline-fix iterations), and every video in the corpus is Indian political content in a mix of English, Hindi, and Urdu. We report it because the numbers are real and, we believe, still informative about where a deterministic-validation, verification-gated-cascade approach actually spends its safety margin – but no accuracy or rate reported here

TABLE I
Corpus and telemetry summary (development-time, not a held-out benchmark)

Distinct source videos (by content hash)	3
Reel ingestion records	5
Extracted claims	28
Distinct claims with ≥ 1 verdict	16
Verdict-generation events (incl. re-runs)	21
Evidence rows analyzed	72
– classified supports	22
– classified contradicts	10
– classified irrelevant	40
Logged search-query failures	55
Logged successful evidence-fetch completions	20
Total tokens logged (input / output)	123,477 / 19,821

should be read as representative of held-out performance. Table I summarizes the raw counts referenced throughout this section; all figures are computed directly from the verdicts, claims, evidence, sources, and audit_logs tables described in Section III.

B. Deterministic validation catch rate

Of 21 verdict-generation events, 15 (71.4%) passed all three deterministic checks unmodified; 6 (28.6%) were downgraded – 5 for an unsupported statistic (a number in the reasoning not found in any cited passage) and 1 for a missing/invalid citation. At the distinct-claim level, 5 of 16 claims (31.3%) had at least one verdict downgraded during testing. We read this as evidence that the validation layer described in Section IV-A is not a redundant formality: on this corpus, it materially changed what was published for roughly three in ten verdict-generation events, every one of which would otherwise have been schema-valid and passed through unflagged.

C. Verification-gated cascade trigger rate

The local model’s output failed the deterministic substantiveness check, triggering escalation to Gemini, in 3 of 7 logged claim-extraction completions (approx. 43%), 2 of 5 content-generation completions (40%), and 2 of 8 verdict-proposal completions (25%). These are small counts and should not be read as a precise escalation rate for the underlying local model in general; they are, however, real, and large enough to show the gate firing routinely rather than rarely – a free-only, no-escalation deployment on this corpus would have shipped several outputs that were schema-valid but substantively empty.

D. Evidence signal-to-noise

Of 72 evidence rows the evidence-analysis stage judged, 40 (55.6%) were classified irrelevant – the fetched source, despite being returned by a search query targeted at the specific claim, did not actually address it. This is consistent with our qualitative finding (Section VI) that search recall, not claim reasoning, is the current bottleneck: over half of retrieved-and-read pages contribute nothing to the eventual verdict.

E. Source tier distribution

Of sources actually retrieved and scored, the majority (54 of 72 scored sources) fell into the lowest other tier (mean reliability 0.404), with only 8 reaching `primary_government` (mean reliability 0.674) and none reaching `primary_legal` or `primary_data` in this corpus, despite the research-planning prompt explicitly requesting at least one primary-source-targeted query per claim. This gap – the prompt asks for primary sources, but the free search backend rarely surfaces them, especially for India-specific government domains – is, in our assessment, the most consequential open problem for output quality, ahead of anything in the LLM reasoning stages themselves; we return to it in Section VIII.

F. Closing the primary-source gap: a deterministic search-side fix

The gap above motivated a concrete fix, mirroring the mechanism already used for `tier3_factcheck` queries (Section IV): a research-planning query explicitly targeted at `tier1_primary` is now restricted at the search-call level itself, via `SearchProvider.search(include_domains=...)`, to a fixed allow-list (`gov.in`, `nic.in`, `sansad.in`, `rbi.org.in`, `indiankanoon.org`) – rather than trusting a free-text `site:gov.in` in the model writes into its own query text, which nothing previously verified actually stayed on those domains. Implementing this surfaced a second, related gap: the source-tier classifier itself was misclassifying genuine government sources as other because its domain hints only matched `.gov` variants – missing NIC (National Informatics Centre), which hosts a large share of official Indian government sites under `nic.in` with no `.gov` substring at all (e.g. the National Testing Agency’s own NEET results page), and missing `rbi.org.in`, the primary source for official Indian financial statistics. Both were added to the classifier alongside the search-side fix.

We then re-ran seven real, unscripted `tier1_primary` queries against the live DuckDuckGo backend (no simulated results) to check whether the restriction actually changes what gets retrieved. Four queries returned results; three returned none at all (a UP-advertising -spending query, a social-media-takedown-rule query, and a second UP -advertising-budget query variant each came back empty – `DDGSEException: No results found`). Pooling the four queries that did return results (20 sources total, after the classifier fix above): 18 `primary_government`, 1 `primary_data`, 1 other – 19 of 20 (95%) landing in a primary tier, against the 8 of 72 (~11%) baseline reported just above. One of the four queries (on India’s social-media intermediary rules) returned the actual MeitY amendment-rules PDF and `egazette.gov.in` directly; another (on state-wise per-capita income) returned five Rajya Sabha-question-citing PIB releases and NITI Aayog/EAC-PM/MoSPI pages, all genuinely on-topic.

Two findings temper this result rather than complicate it away. First, a query about the organization “Karni

Sena” returned five results that were all, correctly, `primary_government`-tier domains, but topically irrelevant – generic government registration portals matched on keyword overlap, not evidence about the organization itself. Domain restriction guarantees tier correctness, not topical relevance; those are separate properties, and this fix addresses only the first. Second, for the per-capita-income query’s five URLs, every full-page-content fetch failed: two 403 Forbidden (`pib.gov.in`, `niti.gov.in`) and three TLS certificate-verification failures (`eacpm.gov.in`, `invest.up.gov.in`, `mospi.gov.in`), falling back to the search backend’s own snippet text (199–336 characters per source). We manually confirmed those snippets were still substantive and on-topic – e.g. real Rajya Sabha Unstarred Question text naming state-wise per capita income figures – so the 95% figure above is not built on empty content, but full source text is being lost specifically for the domain category this fix targets. We did not fix this: doing so would mean either relaxing certificate verification for a curated domain list, a real security trade-off we were not willing to take on without more investigation, or diagnosing the specific 403s further, and we did not want to ship an under-examined fix to a security -relevant check in the same pass as the retrieval fix above. We report it here as a distinct, open limitation rather than folding it into the 95% headline number, which measures tier classification of retrieved results, not the completeness of the text behind them.

G. An out-of-sample pilot

Every number above comes from the same handful of videos used during development. As a first, small step toward the held-out evaluation Section VIII calls for, we ran TruthLens against two reels it had never seen, each independently fact-checked by a professional Indian fact-checking organization (BOOM Live, Alt News), sourced by fetching the organization’s own article and extracting the original Instagram post it had embedded as evidence – not by us selecting or writing the claim or the verdict. On both, TruthLens’s reel-level verdict (`MOSTLY_FALSE` in both cases) agreed with the independent ground truth (`FALSE` in both cases). We do not read this as a validated accuracy result – $n = 2$ – but the process of running it surfaced something more informative than the label match itself.

On the second case, the label match was coincidental rather than earned. The reel’s caption never states the claim that was actually false in words; the misinformation is entirely visual (real World Cup celebration footage captioned as a protest). TruthLens’s text-derived claim extraction found two tangential claims in the caption’s moralizing language and never identified or checked the one thing that was actually misleading: what the video shows. Both pilot cases follow this same pattern – genuine footage, false caption about what it depicts – which was also the dominant pattern among the professional fact-checks we surveyed while sourcing this pilot. TruthLens

has no video-provenance or reverse-image-search capability; its architecture may be structurally weak against the single most common real-world failure mode in this domain, independent of anything about LLM reasoning quality. We treat this as the most important finding of this pilot and return to it in Section VIII.

The same run also surfaced two further bugs, both fixed the same day (Section VI): evidence retrieval confused two similarly-named but distinct organizations, and a verdict the deterministic validator had already downgraded still had its full free-text reasoning reused downstream, carrying an entity-confused, unsupported claim about a real organization’s ideology onto a rendered slide. Neither is visible in the aggregate statistics above, which is itself worth noting: a system can look healthy on its summary numbers while a specific, reproducible, reputationally serious failure sits one level down.

H. A first naive-baseline comparison

Using the same two pilot claims, we ran a genuinely naive baseline: the claim text sent directly to the same LLM TruthLens uses by default (Llama 3.2 via Ollama), with a bare prompt asking for a verdict and reasoning – no claim decomposition, no search, no retrieved evidence, no validation. On both claims the naive baseline answered UNVERIFIABLE, where TruthLens’s full pipeline reached MOSTLY_FALSE (matching ground truth on both, per Section V-G).

We read this narrowly and are wary of overclaiming it. The naive baseline did not reason incorrectly – it correctly recognized it has no way to know about mid-2026 events from training data alone and said so, rather than fabricating an answer. What separates the two columns here is access to current information via search, not any demonstrated advantage in claim decomposition, source tiering, or verdict reasoning specifically. This two-claim comparison mostly shows that having any search access beats having none, which is a much lower bar than validating TruthLens’s specific multi-stage architecture against a simpler competitor that also has search. A fairer ablation – one search query plus a single LLM call over the results, isolating what TruthLens’s multi-stage design adds beyond “search plus one model call” – is needed before this comparison can support a stronger claim, and is now the more precisely scoped version of Section VIII’s baseline item.

I. A second out-of-sample pilot: a photo post, and a grounded correction

A user-supplied Instagram URL turned out to be a single-image post, not a video – TruthLens had no path for that at all, only a hard failure. Building one (Section III-C) and testing it against this real post, an image captioned “The second poorest state by income, but number one in publicity and propaganda” with an overlaid graphic citing specific government ad-spending figures, surfaced

four further real issues before it produced usable output, each fixed and re-verified the same way as Section VI’s taxonomy: an unrelated crash on the zero-verifiable-claims path, on-screen text read correctly by vision analysis but silently dropped before reaching claim extraction, a fabricated headline number (detailed in Section VI), and a missing glyph for the Indian Rupee sign in the rendering font. With those fixed, claim extraction correctly produced two specific, verifiable claims (a rupee figure for government ad spending; a state income ranking) and one correctly -excluded opinion-framed claim.

This post doubled as the first live test of the corrected-fact mechanism (Section IV-E), and it is worth reporting exactly what happened rather than only the eventual success. The claim’s own retrieved evidence already contained a specific correction – an investigative piece citing the state government’s own budget documents for a figure roughly 70% higher than the claim stated. Across three consecutive calls, the default local model’s verdict was downgraded each time, for a real and unrelated reason: its reasoning_summary cited a number that appeared in the evidence matrix but not in the specific source it had formally cited. Per Section IV-E’s design, this correctly suppressed corrected_fact on every one of those three calls, even though the field itself would likely have been fine – the system was conservative in exactly the situation it should be. A call to the fallback model produced a clean pass: verdict MOSTLY_FALSE, and a corrected_fact stating the actual figure with its source, which we confirmed against the underlying article’s own text before trusting it ourselves. Rendering it surfaced one more issue – the new content’s height was undercounted in the slide layout, overflowing one evidence card into the next – fixed and re-verified visually. We read the three-downgrades-then-one-clean-pass pattern as a small but genuine demonstration of Section IV-C’s verification-gated design doing its job under real, not synthetic, conditions: the local model’s own citation imprecision, not a flaw in the new correction mechanism, was what held the output back until a stronger model produced something that actually passed.

J. A third live test: two infrastructure bugs found, no finished output

Not every live test ends in a published fact-check, and we report this one because it did not. A third real Instagram URL – a noisy, multi-screenshot video montage with a heavily code-switched Hindi/Urdu/English transcript – surfaced two further bugs, neither in the LLM reasoning stages this time but in the surrounding pipeline infrastructure.

First, local claim extraction returned two schema-valid claims with text="" – empty strings – for content whose OCR text plainly contained checkable claims (a document authenticity dispute, a funding -source claim). This passed schema validation (an empty string is still a

string) and was invisible to the existing grounding check (Section IV-C), which only ever evaluates verifiable claims’ source_quote field and is a no-op whenever every returned claim comes back non-verifiable, exactly what happened here. This is the same “schema-valid but substantively empty” failure class already guarded against in the verdict and content-generation stages (Section IV-C), just not, until now, in claim extraction itself. Fixed with an analogous substantiveness check that escalates to the fallback model whenever every extracted claim is blank, plus an unconditional filter that never persists a blank-text claim regardless of which model produced it.

Second, with that fix correctly triggering escalation more often, a retry call hit the fallback model’s free-tier daily quota (20 requests per day). The specific API surface this system uses (an interactions-style endpoint, adopted after the older generation endpoint stopped accepting new API keys) raises a completely separate exception hierarchy from the one the provider’s error handling was written against; we confirmed live that the rate-limit exception was not an instance of the caught type at all. The mismatch let a routine, expected failure mode – quota exhaustion – propagate as an unhandled exception instead of the intended typed error, crashing the entire in-flight request after some forty minutes of otherwise-successful work (dozens of completed local-model calls across several claims) rather than failing fast with a clear message. Fixed by catching the actual exception hierarchy the SDK uses; re-verified live post-fix that the identical quota-exhaustion condition now surfaces as a clean, sub-minute error instead of a long hang ending in a crash. Notably, a pre-existing unit test for this code path had mocked the same wrong exception hierarchy the production code was (mis)handling, so it passed throughout despite the underlying bug – a reminder that a test can be internally consistent with buggy code and still provide no real coverage; it was rewritten against the SDK’s actual exception classes.

With the fallback model’s daily quota already spent by the time both fixes were verified, we were not able to complete a finished, fallback-quality fact-check for this specific post the same day – local-only extraction on this content returned zero claims, an honest null result rather than a crash, consistent with the low ceiling on small-model reasoning quality already discussed in Section II and VIII. We report the two infrastructure fixes as real, tested, and independently verified; we do not report a third successful pilot, because there isn’t one yet for this post.

VI. A Taxonomy of Failure Modes Found in Live Testing

Beyond the aggregate statistics above, iterative live testing against a real deployed reel surfaced several specific, reproducible failure modes that we believe are under-documented in the automated fact-checking literature at this level of implementation detail, because they only

appear when a system is actually run end-to-end against real content rather than evaluated on a static benchmark.

- Schema-valid, substantively empty output. Discovered independently in three different pipeline stages (claim extraction, content generation, verdict reasoning): a locally hosted model produces output that satisfies its Pydantic/JSON schema – a non-empty string, a well-typed list – while containing no actual information, e.g. a reasoning_summary composed entirely of [[evidence_id=...]] citation markup with zero prose. No schema validation catches this class of failure; only a content-level substantiveness check does (Section IV-C).
- Verbatim-but-misattributed quotation. A claim’s source_quote can be a genuine, verbatim substring of the video’s on-screen text – passing a naive “is this a real quote” check – while still being the wrong kind of quote: a news-style caption written by the video’s publisher about a person (“X Attacks Y Over Z Rule”), not words that person said. Attributing such a quote to the person it names, because their name appears nearby, produces a fabricated first-person quotation from genuine, verbatim text. We fixed this by requiring the candidate speaker’s own name not to appear inside the quote text itself before trusting the attribution – a person’s own quote essentially never refers to them self by full name.
- Silent verdict/presentation inconsistency. After introducing reel-level, multi-claim verdict aggregation (Section III), the review dashboard continued displaying the primary claim’s own individual verdict as “the” verdict, while the actually-published carousel displayed the separately computed reel-level verdict. The two could and did disagree (one test case: dashboard showed the primary claim’s UN-VERIFIED; the carousel showed the reel’s aggregate MOSTLY_TRUE) with no error, because both values were valid data – the bug was in which one the interface trusted, not in either computation. This class of failure – two correct numbers, wrong place shown – will not be caught by any per-field validation test and was only found by comparing rendered output against the database directly.
- Bracket-matching truncation bug. A display-sanitization regex intended to strip an internal [VALIDATION NOTE: ...] suffix used non-greedy bracket matching, which stopped at the first] character encountered – the closing bracket of a Python list repr inside the note itself (e.g. “Numbers [‘3065258’, ‘69’] in reasoning_summary...”), leaving the note’s own tail visible on a published slide. Fixed by truncating at a literal marker string rather than attempting to bracket-match free-form text.
- Unrepresentative thumbnail selection. A single frame extracted at a hardcoded 0.5-second offset is not

a reliable representative thumbnail; in one case it landed on an unflattering extreme close-up mid-gesture. We now sample several candidate frames across the video’s duration and select by a cheap grayscale-variance “visual complexity” score, screening out blank/flat/transition frames – not real scene- or face-aware selection, but a materially better floor than one fixed guess.

- Entity confusion between similarly-named organizations. Found during the out-of-sample pilot (Section V-G): researching a claim about “Karni Sena,” evidence retrieval returned a Wikipedia article about Sri Ram Sena – a real but different Indian organization sharing the common suffix “Sena.” The evidence-analysis stage labeled this source’s stance as contradicts rather than irrelevant: an article about a different entity does not contradict a claim about this one, it simply does not address it. Name-collision entity disambiguation is not currently checked anywhere in the pipeline.
- Downgraded reasoning reused as trusted input downstream. Also found during the out-of-sample pilot, and the more serious of the two: the verdict-proposal model, given the mismatched Sri Ram Sena evidence above, wrote that this “suggests an unrelated organization with violent ideology” – a real hallucination the deterministic validator correctly downgraded (for an unrelated reason: an unsupported number elsewhere in the same reasoning text). But downgrading a verdict only forces its label to UNVERIFIED; it does not redact the free-text reasoning_summary the model wrote to justify its original, rejected verdict, because there is no reliable way to isolate just the offending sentence from otherwise -reasonable-looking prose. That full text was still being reused verbatim in two places – an evidence card’s displayed answer text, and as raw input material for the separate overall “why” paragraph LLM call, which wove it into the actual rendered slide. This is a real instance of exactly the failure class the validation architecture (Section IV-A) exists to prevent, in a genuine gap in that architecture’s coverage: validation applied to whether a verdict is safe to label, not to whether its reasoning text is safe to quote. Fixed by treating any non-passed validation_status as disqualifying for reuse of that verdict’s free text anywhere, replacing it with a generic, non-fabricated note instead. This never reached a real audience – the fact-check stayed in the internal review queue – but the underlying entity-confusion in evidence retrieval is unfixed and could resurface in a form these particular checks don’t catch.
- Single-digit fabrication in generated headlines. Found during the second pilot (Section V-I): HEADLINE_SYSTEM_PROMPT explicitly instructs the model never to introduce a number not already in the

source claim, and it did so anyway, converting a claim stating “4,000 crore rupees” into a headline reading “\$1 Billion” – both invented and wrong (4,000 crore is roughly \$480M). This is the same failure category as the organization-confusion case above (a free-text generation stage violating an explicit grounding instruction) but in a different pipeline stage, which is itself the point: prompt-level instructions are not self-enforcing, and each generation surface needs its own deterministic check, not a single shared assumption that “we told it not to” is sufficient anywhere. The existing number-grounding check (Section IV-A) ignores single-digit numbers to avoid flagging incidental meta-commentary in longer prose (“3 sources”); a headline is short enough that this exception does not apply, and a naive reuse of that check would have missed “\$1” entirely. Fixed with a headline-specific check requiring every digit sequence in the generated headline to appear in the source claim, falling back to the claim’s own (already-grounded) text verbatim on failure rather than a second model attempt, since a second attempt is not guaranteed not to fabricate a different number.

- Vision-read text silently discarded before claim extraction. Also found during the second pilot: vision analysis of the photo correctly read a specific figure and timeframe off an on-screen graphic into a dedicated “visible text” field, distinct from a separate, vaguer scene-description field – but claim extraction’s prompt only ever included the latter. The specific, checkable content a vision model had already extracted was available in the database and simply never reached the stage that needed it, which is a different class of problem from every other entry in this taxonomy: not a hallucination, but a real signal being thrown away before it could be used. Fixed by including the on-screen-text field with the same weight as OCR output, distinct from the advisory-only scene description.
- Missing glyph rendering as a blank box. The bundled rendering fonts have no glyph for the Indian Rupee sign, which rendered as a placeholder box on a published slide showing a government spending figure – a routine character in this specific domain’s claims, not an edge case. Fixed with a character substitution applied only at render time, leaving the underlying data untouched everywhere else (captions, API responses, the dashboard).
- Vision-transcription accuracy, left unresolved. Distinct from the discarded-field bug above: the same vision analysis occasionally misreads the text it is looking at. One quote box displayed on a published slide read as truncated words (“sp.” for “spent,” “pu” for “publicity”) interrupted by a single stray Devanagari character standing in for a number, where the underlying claim record correctly identified the

same on-screen text as “spent ... 4,000 crore.” This corrupted only the display quote; the claim text used for research and verdicts was unaffected, so it did not change any conclusion, but it is a real, visible defect we chose not to patch with an unverified heuristic (for instance, flagging isolated non-Latin characters in otherwise-Latin text) rather than risk a fix with an unknown false-positive rate. This is qualitatively different from every fixed entry above: an accuracy limitation of the underlying vision model itself, not a gap in the pipeline’s use of it, and it does not have a clean deterministic fix the way a grounding check does.

- Empty-string claims passing schema validation. Found during the third live test (Section V-J): a locally hosted model returned claims with `text=""` for content whose OCR plainly contained checkable claims. A non-empty-string type constraint does not require a non-empty string – the same category of gap as the citation-only reasoning_summary entry above, but in claim extraction rather than verdict reasoning, and invisible to that stage’s existing grounding check specifically because the check only evaluates verifiable claims and treats zero verifiable claims as trivially fine, which is correct for a genuine “nothing here” extraction but blind to “something here, just empty.” Fixed with a dedicated substantiveness check (Section IV-C) plus an unconditional filter that never persists a blank-text claim regardless of which model produced it.
- An except clause written against the wrong SDK exception hierarchy. Also found during the third live test: escalating to the fallback model correctly triggered more often after the fix above, which surfaced a second, unrelated bug – a real rate-limit response from the fallback provider’s API raised an exception type the provider wrapper’s error handling did not recognize as an instance of the class it was written to catch, because the specific API surface in use (adopted after an older endpoint stopped accepting new API keys) turns out to raise a separate exception hierarchy entirely. The mismatch let an entirely routine, expected condition – daily quota exhaustion – propagate as an unhandled exception rather than the intended typed error, crashing an in-flight request after roughly forty minutes of otherwise-successful work instead of failing fast with a clear message. A pre-existing unit test for this exact code path had mocked the same wrong exception hierarchy the production code was (mis)handling, so it passed throughout – a concrete instance of a broader risk worth naming: a test can be internally consistent with buggy code, agreeing with a wrong assumption on both sides, and still provide zero real coverage of the failure it was written to catch. Fixed by catching the SDK’s actual exception hierarchy and rewriting

the test against real instances of it rather than hand-constructed doubles of the wrong type.

We consider this taxonomy, and the general lesson that most of these failures were only discoverable by running the full system against real content and inspecting rendered output rather than by unit-testing individual functions in isolation, to be as practically important a contribution of this work as any single architectural component.

VII. Discussion and Threats to Validity

Small, non-independent sample. The telemetry in Section V is drawn from repeated testing against a handful of real videos, not independent draws from a target population; several statistics (e.g. the 28.6% validation-downgrade rate) could shift substantially with a larger, held-out corpus. We report exact counts, not just percentages, throughout so a reader can judge this directly.

Single domain, single language mix. All evaluated content is Indian political short-form video with English/Hindi/Urdu mixed transcripts. Source-tiering domain lists and the research-planning prompt’s primary-source guidance were not tuned for, and have not been tested against, other national contexts.

No systematic bias audit. Section II discusses documented partisan asymmetry in frontier LLMs; TruthLens mitigates this only via prompt-level instruction (the neutrality clause), which is not verified against a labeled bias-evaluation set.

Baseline comparison is preliminary and too coarse. We ran a naive single-shot baseline on the two pilot claims (Section V-H), but at $n = 2$, and in a form that mostly isolates “has search access” rather than “has TruthLens’s specific multi-stage architecture.” A baseline that also has search – disentangling the value of retrieval itself from the value of TruthLens’s decomposition/tiering/validation design – is still needed and is the most direct way to substantiate the paper’s central architectural claim.

Grounded corrections and photo-post ingestion are each tested on essentially one real case. Section IV-E’s corrected_fact mechanism produced a clean, verified result exactly once, after being correctly suppressed on three prior calls for an unrelated reason (Section V-I); how often it actually fires with a specific, grounded correction on a broader corpus – as opposed to correctly returning null when the evidence does not establish one – is unknown. Photo-post ingestion (Section III-C) is similarly tested against a single real post. Both are real, verified, working code paths, not simulated, but neither has been exercised at anything like the scale needed to report a meaningful success rate.

The primary-source fix is measured on tier classification, not on full-text completeness. Section V-F’s 95% figure describes what tier the retrieved sources were classified into, across a pool of only 20 sources from 4 successful queries out of 7 attempted. It does not describe whether

the system retrieved those sources’ full text: for one of the four queries, every page fetch failed (403s and TLS certificate errors) and the pipeline fell back to search-snippet text throughout. We manually spot-checked that the snippets were substantive, but this was not verified systematically, and the underlying .gov.in fetch-reliability gap is unresolved, not merely undiscussed.

The third live test’s fixes are infrastructure-hardening, not a new capability, and are each validated by one live failure and one live re-verification. Section V-J’s two fixes (blank-claim substantiveness checking; the corrected exception hierarchy) were each triggered exactly once by real conditions and confirmed fixed by reproducing the same condition once more post-fix – not exercised across a range of inputs. The daily quota limit on the fallback provider also means this specific failure mode (a rate-limit exception escaping unhandled) is, by construction, rare to trigger deliberately without either exhausting real quota or mocking the SDK, which is what the rewritten unit test now does; live re-triggering was possible here only because a real pipeline run happened to exhaust the quota during testing, not because we engineered the condition on demand.

VIII. Future Work

- 1) A held-out, labeled benchmark. Curate a labeled set of short-form political video claims, disjoint from any content used during development, with ground-truth verdicts from independent human reviewers (ideally with inter-annotator agreement reported), and report precision/recall/F1 against it – the single change that would most improve this paper’s evidentiary standing.
- 2) A search-equipped baseline comparison. Our first attempt (Section V-H) used a baseline with no search access at all, which mostly showed that search access matters – unsurprising, and not the comparison that actually matters most. The needed version gives the baseline one web search and a single LLM call over the raw results, holding retrieval access constant so the comparison isolates what TruthLens’s multi-stage decomposition, source tiering, and deterministic validation specifically add beyond “search plus one model call.”
- 3) Video-provenance verification. The out-of-sample pilot (Section V-G) found both tested cases, and the majority of professional fact-checks surveyed while sourcing it, follow the same pattern: genuine footage recaptioned with a false claim about what it shows or where it’s from. TruthLens has no mechanism to check this – claim extraction works from transcript/OCR/caption text, and a purely-visual false claim (wrong location, wrong event, wrong date) is invisible to it regardless of how good the downstream reasoning is. Reverse-image/video search against known reference footage, or at minimum flagging

when a claim about a video’s provenance cannot be checked from text alone (rather than silently checking a different, tangential claim instead), would target what the pilot suggests is the dominant real-world failure mode in this domain – likely higher leverage than any of the LLM-reasoning-focused items below.

- 4) Vision-transcription accuracy. Section VI documents a case where vision analysis of an on-screen graphic introduced a stray character and truncated words into an otherwise -correct reading. This did not affect any conclusion in that instance (the separately-extracted claim text was correct), but it is an accuracy limitation of the underlying vision model itself, not something a grounding check can catch the way a fabricated number can – the text was present, just imprecisely transcribed. Systematically measuring this rate (e.g. character-level edit distance between vision-read text and a human transcription on a small sample) would clarify whether it is rare noise or frequent enough to need a dedicated mitigation.
- 5) Primary-source retrieval – substantially addressed, one gap remaining. The domain-allow-list fix this item originally called for is now implemented and live-tested (Section V-F): a dedicated tier1_primary query restricted to government/legal domains at the search-call level, plus a related source-classifier fix (nic.in, rbi.org.in), moved pooled primary-tier retrieval from 8 of 72 (~11%) to 19 of 20 (95%) across four real queries – with the honest caveats that 3 of 7 real queries returned no results at all, and that domain-correctness does not imply topical relevance (the “Karni Sena” case). What remains open: a page-fetch reliability gap specific to .gov.in domains (403s and TLS certificate failures on several ministry/state sites, currently masked by falling back to search-snippet text), which needs either a deliberately scoped certificate-verification exception or further diagnosis of the specific failures, not attempted this pass.
- 6) Systematic bias evaluation. Apply an existing partisan- asymmetry evaluation methodology (Section II) to TruthLens’s own verdicts across politically matched claim pairs.
- 7) Cross-lingual and cross-domain generalization. Test source tiering and research-planning prompts against non-Indian political content and languages beyond the current English/Hindi/Urdu mix.
- 8) Calibration. Compare TruthLens’s reported confidence bands against empirical correctness rates once a labeled benchmark exists (Item 1), to check whether “very high confidence” verdicts are in fact more often correct than “moderate confidence” ones.

IX. Conclusion

TruthLens demonstrates that a substantial fraction of the “does the model hallucinate” problem in LLM-based fact-checking is addressable architecturally rather than only through better prompting or a more capable model: a deterministic, non-LLM validation layer applied after generation caught 28.6% of generated verdicts in our development corpus, and an explicit distinction between genuine null results and infrastructure failure prevents an entire class of false confidence. These results come from a small, non-independent development corpus and should be read as preliminary telemetry, not a benchmark evaluation – but the failure modes they surfaced (schema-valid-but-empty output, verbatim-but-misattributed quotation, silent presentation inconsistency, single-digit numeric fabrication in a generated headline, a correctly-extracted signal silently discarded before it reached the stage that needed it) are, we believe, general enough to matter to any team building an LLM pipeline whose output will be published and trusted – and general enough, in the case of the headline fabrication, to recur in a newly added feature (photo-post ingestion) despite the underlying validation architecture already existing, because grounding has to be re-applied at every new generation surface rather than assumed to transfer. Extending the same validation methodology from verdict labels to grounded corrections (Section IV-E) is a first step toward the kind of output actually useful to a reader – not just “false,” but what the evidence shows instead – while the explicit decision not to extend it into predicting consequences marks where we believe grounded fact-checking has to stop and speculation would begin. Closing the primary-source retrieval gap our own evaluation had flagged (Section V-F) shows the same pattern at the retrieval stage rather than the generation stage: a deterministic, already-proven mechanism (domain-restricted search) applied to a previously-unenforced prompt-level request moved primary-tier retrieval from ~11% to 95% in live testing, while honestly surfacing a distinct, unresolved reliability gap in fetching full page text from .gov.in domains specifically – a reminder that fixing one measured gap tends to reveal the next one, not eliminate the need for further measurement. A third live test (Section V-J) found the same “schema-valid-but-empty” pattern recurring in a third pipeline stage (claim extraction, after verdict reasoning and content generation) despite two prior fixes for the identical failure class – reinforcing that grounding checks do not generalize across stages automatically – and, separately, a wrong-exception-hierarchy bug that a same-purpose unit test had not caught because it mocked the identical wrong assumption the production code made; both are fixed, though the specific post that surfaced them still has no finished fact-check, reported here rather than quietly dropped. The most direct path to strengthening every claim in this paper is the held-out benchmark

and search-equipped baseline comparison described in Section VIII, which we intend to pursue as this work continues.

References

- [1] L. Shang, Z. Kou, Y. Zhang, and D. Wang, “A multimodal misinformation detector for COVID-19 short videos on TikTok,” in 2021 IEEE International Conference on Big Data (Big Data), 2021.
- [2] J. Thorne, A. Vlachos, C. Christodoulopoulos, and A. Mittal, “FEVER: a large-scale dataset for fact extraction and VERification,” in Proceedings of the 2018 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies (NAACL-HLT), 2018.
- [3] W. Y. Wang, “Liar, Liar Pants on Fire”: A new benchmark dataset for fake news detection,” in Proceedings of the 55th Annual Meeting of the Association for Computational Linguistics (Volume 2: Short Papers), Vancouver, Canada, 2017.
- [4] S. S. Rahman, M. A. Islam, M. M. Alam, M. Zeba, M. A. Rahman, S. S. Chowdhury, M. A. K. Raiaan, and S. Azam, “Hallucination to truth: A review of fact-checking and factuality evaluation in large language models,” *Artificial Intelligence Review* (Springer); arXiv preprint arXiv:2508.03860, 2025.
- [5] I. Vykopal, M. Pikuliak, S. Ostermann, and M. Šimko, “Generative large language models in automated fact-checking: A survey,” arXiv preprint arXiv:2407.02351, 2024.
- [6] R. Panchendrarajan and A. Zubiaga, “Claim detection for automated fact-checking: A survey on monolingual, multilingual and cross-lingual research,” arXiv preprint arXiv:2401.11969, 2024.
- [7] N. Hassan, F. Arslan, C. Li, and M. Tremayne, “Toward automated fact-checking: Detecting check-worthy factual claims by ClaimBuster,” in Proceedings of the 23rd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining (KDD), 2017, pp. 1803–1812.
- [8] V. Magesh, F. Surani, M. Dahl, M. Suzgun, C. D. Manning, and D. E. Ho, “Hallucination-free? assessing the reliability of leading AI legal research tools,” arXiv preprint arXiv:2405.20362, 2024, preregistered, hand-scored (202 queries) evaluation finding leading commercial legal AI tools hallucinate 17–33% of the time.
- [9] K. Khajavi, S. Sadeghi, R. Adhikari, and A. Tessier, “CiteCheck: Retrieval-grounded detection of LLM citation hallucinations in scientific text,” arXiv preprint arXiv:2605.27700, 2026.
- [10] V. Ovcharov, “Citation grounding measures the oracle: Graph coverage determines reported LLM hallucination rates in law,” arXiv preprint arXiv:2606.00898, 2026, shows apparent citation-hallucination rates from graph-based oracles (15–21% on a sparse snapshot) are largely an artifact of citation-graph coverage rather than a stable property of the model being measured.
- [11] M. R. DeVerna, K.-C. Yang, H. Y. Yan, and F. Menczer, “Large language models require curated context for reliable political fact-checking – even with reasoning and web search,” arXiv preprint arXiv:2511.18749, 2025.
- [12] H. Vatndal and V. Setty, “ShortCheck: Checkworthiness detection of multilingual short-form videos,” in Proceedings of the 14th International Joint Conference on Natural Language Processing and the 4th Conference of the Asia-Pacific Chapter of the Association for Computational Linguistics: System Demonstrations, Mumbai, India, 2025.
- [13] T.-Q. Peng, K. Yang, S. Lee, H. Li, Y. Chu, Y. Lin, and H. Liu, “Beyond partisan leaning: A comparative analysis of political bias in large language models,” *Journal of Information Technology & Politics*, 2026.
- [14] L. Chen, M. Zaharia, and J. Zou, “FrugalGPT: How to use large language models while reducing cost and improving performance,” arXiv preprint arXiv:2305.05176, 2023.